

SIS Project Guidelines

Golang Application Development. KBTU. FALL 2026

The SIS component is an **individual cumulative programming project** developed throughout the semester.

The goal of the project is to apply the Go concepts studied during the course in a single evolving application. The project is divided into three checkpoints, each worth **5 points**, for a total of **15 points**.

Students are responsible for understanding all submitted code and must be able to explain their implementation during an individual code defense.

The application will gradually evolve from a simple in-memory Go program into a concurrent application with a minimal HTTP interface.

Checkpoint 1 - Core Golang.....	2
Checkpoint 2 - Concurrent Processing.....	3
Checkpoint 3 - Concurrent Processing.....	4
Submission.....	5
Individual Code Defense.....	5
Assessment.....	6

Checkpoint 1 - Core Golang

Points: 5

The first checkpoint focuses on the core Go language concepts covered during Weeks 1-7.

The project must include:

- A clear domain model for tasks and their states
- Appropriate use of structs and methods
- Appropriate use of interfaces
- In-memory storage using Go data structures
- Proper error handling
- Unit tests for the core functionality

Concurrency is **not required** at this stage.

The student should be able to explain decisions related to value and pointer semantics, receivers, interfaces, slices/maps, and error handling.

Checkpoint 2 - Concurrent Processing

Points: 5

The second checkpoint extends the existing project with concurrent task processing.

The project must include:

- Multiple worker goroutines
- A bounded task queue
- Appropriate synchronization of shared state
- Communication through channels where appropriate
- Controlled worker lifecycle
- Correct handling of concurrent access

The implementation must pass:

```
go test ./...  
go test -race ./...
```

The student should be able to explain the lifecycle of each goroutine, ownership of channels, synchronization decisions, and possible race or deadlock scenarios.

Checkpoint 3 - Concurrent Processing

Maximum: 5 points

The final checkpoint integrates the previously implemented components into a complete application.

The project must include:

- Context-based cancellation and timeouts
- A minimal HTTP API using `net/http`
- Graceful shutdown
- Automated tests
- Correct error handling
- Clean and maintainable Go code

At minimum, the application should provide operations for creating a task and retrieving its current state or result.

The student should be able to explain how requests propagate through the application, how cancellation works, how concurrent components terminate, and why the chosen design is safe.

Submission

Each checkpoint must be submitted before the announced deadline.

The code submitted before the deadline is the version evaluated during the defense. Changes made after the deadline may not be considered.

The project must compile and run according to the instructions provided with the assignment.

Students must submit their own individual work.

Students maintain an individual GitHub repository throughout the semester. For each checkpoint, they submit the repository link and the corresponding commit/link/tag through Microsoft Teams before the deadline.

Individual Code Defense

Each checkpoint includes a short individual code defense conducted during the practical session.

The defense normally lasts approximately **2 minutes**. There is no presentation and no slides.

During the defense, you may be asked to:

- Explain a selected part of your implementation
- Justify a particular Go construct or design decision
- Explain what would happen in a particular edge case
- Discuss an alternative implementation
- Identify a potential correctness or concurrency problem

Questions will be based primarily on your submitted code. You are expected to understand and be able to explain **all parts of your solution**, not only the parts you expect to be asked about.

The defense is mandatory. **If you submit the checkpoint but do not complete the defense, you receive 0 points for that checkpoint.**

Assessment

Each checkpoint is worth a maximum of 5 points.

- Implementation and functional correctness - up to 2.5 points
- Individual code defense - up to 2.5 points

To receive **5/5**, the submitted implementation must:

- satisfy all checkpoint requirements;
- compile successfully;
- pass all mandatory tests;
- demonstrate correct use of the required Go concepts;
- be successfully defended.

The individual code defense is **mandatory**. If you submit the checkpoint but do not complete the defense, you receive **0/5** for that checkpoint.